

Introduction to the Microsoft Identity Platform and Entra ID

Rob Windsor
rob@robwindsor.com
@robwindsor

About Me



Rob Windsor

.NET/Microsoft 365 developer, trainer, author
Microsoft MVP | Microsoft 365 Development



Twitter: <https://twitter.com/robwindsor>

LinkedIn: <https://www.linkedin.com/in/rwindsor>

Blog: <https://robwindsor.hashnode.dev>

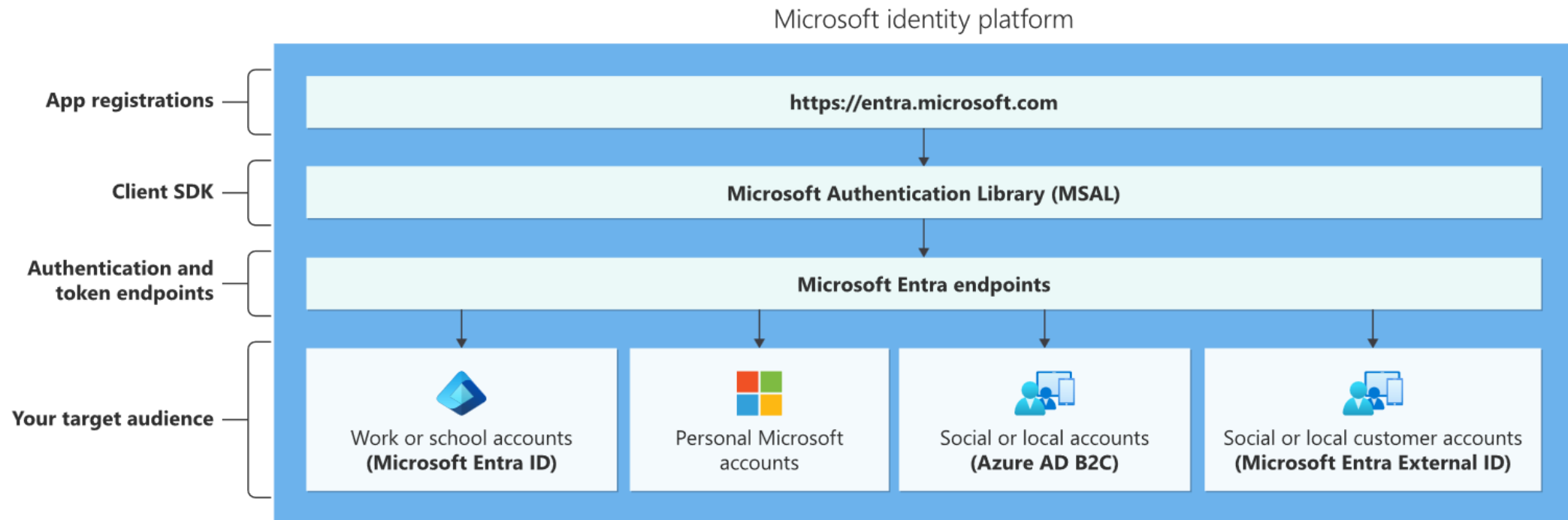
YouTube: <https://www.youtube.com/@RobWindsor>

GitHub: <https://github.com/rob-windsor>

Microsoft Identity Platform

Microsoft's cloud-based identity and access management service built on Entra ID

Single platform that handles both authentication and authorization across M365, Azure, and custom applications



Microsoft Identity Platform

Enables you to build applications that:

- Sign in users with Microsoft work/school or personal accounts
- Sign in users with social accounts (Google, Facebook) via Azure AD B2C
- Authorize access to your own APIs
- Authorize access to Microsoft APIs such as Microsoft Graph

Built on open standards

- OAuth 2.0, OpenID Connect, SAML

Key Concepts

Tenant

- Organization's dedicated instance of Entra ID
- Every Microsoft 365 organization has a tenant

User

- Person with an identity managed in the tenant

Application

- Registered in Entra ID to participate in authentication

Service Principal

- Application's identity within a specific tenant

Managed Identity

- Service principal whose credentials are managed by Azure

Groups and Roles

- Used to manage access to resources and APIs

Entra ID Authentication

Entra ID is the identity provider behind the Microsoft Identity Platform

Authentication verifies who you are

Authorization determines what you can access

Multiple authentication APIs available to handle token acquisition and exchange

Registering an Application

All applications must be registered in the Azure Portal or via CLI/API

- Based on OAuth 2.0

Supported account types:

- Single tenant: only accounts in your organization
- Multi-tenant: accounts in any organization
- Personal Microsoft accounts: consumer accounts (e.g. Outlook.com)

Redirect URIs

- Where the identity platform is allowed to send tokens after authentication

Application Components

- Tenant ID: unique ID for the tenant
- Client (Application) ID: unique ID for the application
- Client Secret or Certificate: credential an application uses to prove its identity
- Redirect URI: whitelist of locations to which Entra ID is allowed to send auth responses
- Permission Requests: requested permissions to resources or APIs
 - Must be granted by a user or an administrator

Static Permission Requests

Permission types

- Delegated: app acts on behalf of a signed-in user
- Application: app acts as itself with no user present

Consent

- User consent: user grants delegated permissions to the app
- Admin consent: administrator grants permissions on behalf of all users in tenant
- Some permissions require admin consent regardless of who is signing in
- Permissions are embedded in the access token as scope (delegated) or roles (app) claims

Static Permission Requests

Dashboard > robwindstest986 | App registrations > InteractiveConsole | API permissions

InteractiveConsole | API permissions

Search (Ctrl+/)

Refresh

Overview

Quickstart

Manage

Branding

Authentication

Certificates & secrets

Token configuration (preview)

API permissions

Expose an API

Owners

Roles and administrators (Previ...

Manifest

Support + Troubleshooting

Troubleshooting

New support request

Configured permissions

Applications are authorized to call APIs when they are granted per all the permissions the application needs. [Learn more about permi](#)

+ Add a permission

Grant admin consent for robw

API / Permissions name	Type	Descripti
Microsoft Graph (1)		
User.Read	Delegated	Sign in an

Request API permissions

[All APIs](#)

Microsoft Graph

<https://graph.microsoft.com/> [Docs](#)

What type of permissions does your application require?

Delegated permissions

Your application needs to access the API as the signed-in user.

Application permissions

Your application runs as a background service or daemon without a signed-in user.

Select permissions

expand all

Mail.Read ✓

Permission	Admin consent required
☒ Mail (1)	
☒ Mail.Read Read user mail ⓘ	-
☐ Mail.Read.Shared Read user and shared mail ⓘ	-
☐ Mail.ReadBasic Read user basic mail ⓘ	-
☐ Mail.ReadWrite Read and write access to user mail ⓘ	-
☐ Mail.ReadWrite.Shared	-

Add permissions

Discard

Dynamic Permission Requests

Only available when using delegated permissions

Permissions requests declared in code - not in the portal

- Requested at runtime by passing scopes to the authentication library

Supports incremental consent

- Request additional permission scopes as the user accesses more features

Tokens

Tokens are JSON Web Tokens (JWTs, pronounced "jots")

Sites like jwt.ms can be used to view token contents

ID tokens

- Enable clients to get identity and profile information for a user
- Should not be used for authorization

Access tokens

- Proves what the app is allowed to do; sent to resource APIs
- Contains authorization claims like audience, scopes, and roles
- Generally short lived

Refresh tokens

- Long-lived token used to acquire new access tokens silently

APIs and SDKs

Microsoft Authentication Library (MSAL)

- General-purpose library for acquiring tokens from the Microsoft Identity Platform
- Lower-level: closer to OAuth2 and OpenID Connect protocols.

Azure Identity Client Library

- Simplified authentication for Azure SDK clients (Storage, Key Vault, etc.)
- Higher-level: hides the complexity of authentication from the application

Microsoft Identity Web

- Recommended library for ASP.NET Core web apps and web APIs
- Wraps MSAL.NET and integrates with ASP.NET Core middleware and dependency injection
- Handles token validation in APIs automatically via **AddMicrosoftIdentityWebApi()**

Microsoft Authentication Library (MSAL)

Recommended library for acquiring tokens from the Microsoft Identity Platform

Available for: .NET, JavaScript/TypeScript, Python, Java, Go, iOS, Android

Two application types:

- `PublicClientApplication`: interactive flows where users are present (desktop, mobile, CLI)
- `ConfidentialClientApplication`: server-side flows with secrets/certs (web apps, APIs, services)

Built-in token cache with silent acquisition

- Checks the cache before making any network call
- If a valid access token exists, it is returned immediately
- If expired, uses refresh token to acquire a new access token silently

NuGet: `Microsoft.Identity.Client`

Azure Identity Library

Higher-level abstraction built on top of MSAL

Good choice for Azure-hosted apps

Managed Identities give Azure resources an Entra ID identity with no secrets to manage

DefaultAzureCredential automatically uses Managed Identity in Azure and falls back to developer credentials locally. This lets you use the same code in both environments

NuGet: Azure.Identity

Authentication Flows

The Microsoft Identity Platform supports several OAuth 2.0 / OpenID Connect flows

Choosing the right flow depends on your app type and use case

Flow	App Types	User	Tokens
Authorization Code	Web apps, SPAs, mobile	Yes	ID, access, and refresh
Client Credentials	Daemons, services	No	Access
Device Code	CLIs, IoT, browserless	Yes	ID, access, and refresh
On-behalf Of	Middle tier	Upstream	Access

Authorization Code Flow

Used in user-facing applications like web apps, SPAs, mobile apps, and desktop apps

Can only use delegated permissions

Steps:

- GET request made to Entra ID authorization endpoint
 - Request includes the client ID and redirect URI on the query string
- User authenticates and consents to permission requests
- Entra ID generates an authorization code
- Entra ID redirects to redirect URI with authorization code on the query string
- POST request made to Entra ID token endpoint to exchange authorization code for tokens
 - Body of request includes: auth code, client ID, and client secret/certificate
 - Response includes an access token and a refresh token

Authorization Code Flow with PKCE (Proof Key for Code Exchange)

Variant of the Authorization Code Flow. Replaces the now-deprecated Implicit Flow, which returned tokens directly in the URL fragment

- Prevents authorization code interception attacks
- Client application generates a random string known as a code verifier
- Client application also generates a code challenge which is a hash of the code verifier
- Request to get an authorization code includes the code challenge
- Request to exchange authorization code for tokens includes code verifier
- Entra ID ensures the code verifier matches the code challenge before returning tokens

Client Credentials Flow

Used in background services, daemons, API-to-API calls

No user authentication – application acts as itself

Can only use application permissions

Steps:

- POST request made to Entra ID token endpoint to get tokens
 - Body of request includes client ID and client secret/certificate
 - Response includes access token only (i.e. no refresh token)

On-behalf-of (OBO) Flow

Used when you have a middle-tier API that needs to call a downstream API on behalf of the original user, preserving their identity and permissions throughout the chain.

Steps:

- User signs in to a client app and receives an access token for a middle-tier API
 - API receives that token in an Authorization header
- Middle-tier API exchanges user token for downstream API token
- Middle-tier API calls downstream API on behalf of the user
 - These calls use the identity and permissions of the user

Validating Tokens

APIs should validate incoming access tokens

Key claims to validate:

- aud: must match your API's client ID (rejects tokens intended for other APIs)
- iss: must match the expected Entra ID tenant or the common endpoint
- exp: token must not be expired
- scp or roles: confirm the required permission is present

Microsoft.Identity.Web handles validation automatically. For other stacks, standard JWT middleware handles signature and expiry; you are responsible for checking aud, iss, and scp/roles

Exposing an API

To secure your own API with Entra ID, you must expose it by defining the permissions other apps can request. This is part of an application registration configuration.

- Application ID URI: unique identifier for your API
 - Set in the "Expose an API" section of your app registration
- Scopes: define delegated permissions your API exposes (e.g. Forecast.Read)
 - Appear as scp claims in access tokens
- App roles: define application permissions your API exposes for app-only access
 - Appear as roles claims in access tokens
- Authorized client applications: optionally pre-authorize specific client apps to skip user consent

Client apps request your custom scopes the same way they request Microsoft Graph scopes — via static or dynamic permission requests

Resources

Microsoft identity platform documentation

- learn.microsoft.com/en-us/entra/identity-platform/

Implement user authentication and authorization

- learn.microsoft.com/en-us/training/paths/az-204-implement-authentication-authorization/

Microsoft identity platform code samples

- learn.microsoft.com/en-us/entra/identity-platform/sample-v2-code